Programming with Python



10. Exception Handling

- ✓ Errors and Exceptions
- ✓ Python Exception Classes
- ✓ Handling Exceptions
- ✓ Various Clauses – try, except, finally, else
- ✓ Examples

Errors and Exceptions

- There are (at least) two distinguishable kinds of errors:
  - *syntax errors*
  - *Exceptions*

- Syntax Errors
  - Syntax errors, also known as parsing errors

```
>>> while True print('Hello world')
  File "<stdin>", line 1, in ?
    while True print('Hello world')
                     ^
SyntaxError: invalid syntax
```

- The parser repeats the offending line and displays a little ‘arrow’ pointing at the earliest point in the line where the error was detected.

Exceptions

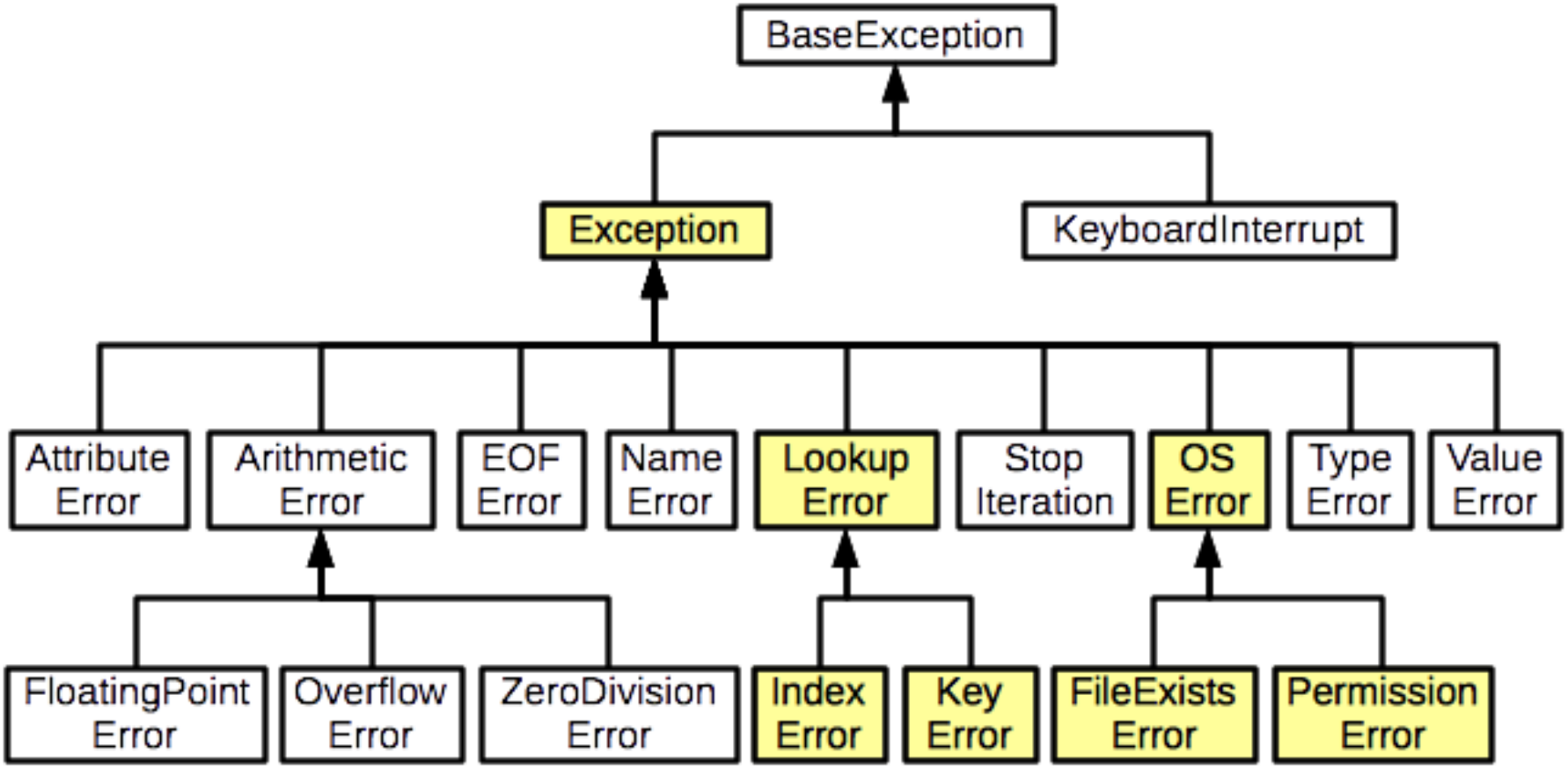
- Exceptions
  - Even if a statement or expression is syntactically correct, it may cause an error when an attempt is made to execute it.
  - Errors detected during execution are called ***exceptions*** and are not unconditionally fatal.
  - Most exceptions are not handled by programs, however, and result in error messages as shown here:

```
>>> 10 * (1/0)
Traceback (most recent call last):
  File "<stdin>", line 1, in ?
ZeroDivisionError: division by zero
>>> 4 + spam*3
Traceback (most recent call last):
  File "<stdin>", line 1, in ?
NameError: name 'spam' is not defined
>>> '2' + 2
Traceback (most recent call last):
  File "<stdin>", line 1, in ?
TypeError: Can't convert 'int' object to str implicitly
```

The string printed as the exception type is the name of the built-in exception.

Python Exception Classes

Exceptions come in different types, and the type is printed as part of the message:



Ref: w3.csjmu.edu

Handling Exceptions

- If you have some *suspicious* code that may raise an exception, you can defend your program by placing the suspicious code in a **try:** block.
- **After the try:** block, include an **except:** statement,
- followed by a block of code which handles the problem as elegantly as possible.
- Syntax:

```
try:
   You do your operations here;
   ......................
except ExceptionI:
   If there is ExceptionI, then execute this block.
except ExceptionII:
   If there is ExceptionII, then execute this block.
   ......................
else:
   If there is no exception then execute this block.
```

Example – Keyboard Interrupt Exception

- Ask the user for input until a valid integer has been entered,
- Allows the user to interrupt the program (using Control-C)
- A user-generated interruption is signalled by raising the KeyboardInterrupt exception.Syntax:

```
>>> while True:
...     try:
...         x = int(input("Please enter a number: "))
...         break
...     except ValueError:
...         print("Oops!  That was no valid number.  Try again...")
...
```

How “try” clause works?

- **First**, the try clause (the statement(s) between the try and except keywords) is executed.
- **If no exception occurs**, the except clause is skipped and execution of the try statement is finished.
- **If an exception occurs** during execution of the try clause, the rest of the clause is skipped. Then if its type matches the exception named after the except keyword, the except clause is executed.
- **If an exception occurs which does not match the exception named in the except clause,** it is passed on to outer try statements; if no handler is found, it is an unhandled exception and execution stops .

How “except” clause works?

- An except clause may name multiple exceptions as a parenthesized tuple
- The last except clause may omit the exception name(s), to serve as a wildcard/default exception.
  - Use this with extreme caution, since it is easy to mask a real programming error in this way!
- It can also be used to print an error message and then re-raise the exception (allowing a caller to handle the exception as well):

“except” clause with multiple exceptions

An except clause may name multiple exceptions as a parenthesized tuple

```
import sys

try:
    f = open('myfile.txt')
    s = f.readline()
    i = int(s.strip())
except OSError as err:
    print("OS error: {0}".format(err))
except ValueError:
    print("Could not convert data to an integer.")
except:
    print("Unexpected error:", sys.exc_info()[0])
    raise
```

“else” clause – Optional

- The try ... except statement has an optional else clause, which, when present, must follow all except clauses.
- It is useful for code that must be executed if the try clause does not raise an exception. For example:

```
for arg in sys.argv[1:]:
    try:
        f = open(arg, 'r')
    except IOError:
        print('cannot open', arg)
    else:
        print(arg, 'has', len(f.readlines()), 'lines')
        f.close()
```

Raising Exceptions

- **raise statement** allows us to force a specified exception to occur.
- The sole argument to raise indicates the exception to be raised.
- This must be either an exception instance or an exception class (a class that derives from Exception).

```
def functionName( level ):
   if level < 1:
      raise "Invalid level!", level
      # The code below to this would not be executed
      # if we raise the exception
```

```
try:
   Business Logic here...
except "Invalid level!":
   Exception handling here...
else:
   Rest of the code here...
```

Clean Up Actions – “try-finally” clause

- You can use a **finally:** block along with a **try:** block.
- The finally block is a place to put any code that must execute, whether the try-block raised an exception or not.

Syntax:

```
try:
   You do your operations here;
   ......................
   Due to any exception, this may be skipped.
finally:
   This would always be executed.
   ......................
```

“try-finally” clause – Example1

- When an exception is thrown in the try block, the execution immediately passes to the finally block.
- After all the statements in the finally block are executed, the exception is raised again and is handled in the except statements if present in the next higher layer of the try-except statement.

```
#!/usr/bin/python

try:
   fh = open("testfile", "w")
   try:
      fh.write("This is my test file for exception handling!!")
   finally:
      print "Going to close the file"
      fh.close()
except IOError:
   print "Error: can\'t find file or read data"
```

“try-finally” clause – Example2

```
>>> def divide(x, y):
...     try:
...         result = x / y
...     except ZeroDivisionError:
...         print("division by zero!")
...     else:
...         print("result is", result)
...     finally:
...         print("executing finally clause")
...
>>> divide(2, 1)
result is 2.0
executing finally clause
>>> divide(2, 0)
division by zero!
executing finally clause
>>> divide("2", "1")
executing finally clause
Traceback (most recent call last):
  File "<stdin>", line 1, in ?
  File "<stdin>", line 3, in divide
TypeError: unsupported operand type(s) for /: 'str' and 'str'
```

Thank You